TUGAS KELOMPOK

IMPLEMENTASI SISTEM CLIENT-SERVER SEDERHANA MENGGUNAKAN SOCKET PYTHON

MATA KULIAH SISTEM KOMPUTER DAN JARINGAN

KELOMPOK 4

Penyusun :

Fathiyah Audyna Ramadhani (23/514605/PA/21995)

Andita Yudha Astuti (23/523214/PA/22512)

Faisha Aldila Putri (23/520308/PA/22366)

Wan Mohammad Renjiro Anza (22/497963/PA/21471)

PROGRAM STUDI ELEKTRONIKA DAN INSTRUMENTASI

DEPARTEMEN ILMU KOMPUTER DAN ELEKTRONIKA

FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM

UNIVERSITAS GADJAH MADA

YOGYAKARTA

2026

I. Penjelasan Arsitektur Sistem

Sistem mini chat sederhana yang dikembangkan menggunakan arsitektur Client–Server berbasis TCP Socket.

1. **Server**

   Server bertugas untuk:

   - Melakukan bind ke alamat IP dan port tertentu.
   - Masuk ke mode listen (menunggu koneksi).
   - Menerima koneksi dari client.
   - Menerima satu perintah dari client.
   - Memproses perintah.
   - Mengirimkan respons ke client.
   - Menutup koneksi.
   - Kembali ke mode standby.

   Server menggunakan:

   - socket.AF_INET
   - socket.SOCK_STREAM (TCP)
   - Threading untuk melayani banyak client secara paralel.
   - Penyimpanan pesan dalam memori (list global MESSAGES).
   - Logging aktivitas ke file (chat.log).

2. **Client**

   Client bertugas untuk:

   - Menghubungkan diri ke server melalui IP dan port.
   - Menerima input dari user.
   - Mengirim perintah ke server.
   - Menerima respons dari server.
   - Menutup koneksi setelah respons diterima.

Diagram Sederhana Arsitektur:

Client → TCP Socket → Server → (Memory + Log File)

Server dapat melayani beberapa client secara paralel menggunakan thread.

II. Flow Komunikasi Data

A. Kirim Pesan (MSG)

1. Client melakukan connect() ke server.
2. Client mengirim perintah: MSG <username> <message>
3. Server menerima data melalui recv().
4. Server:
   - Parsing perintah.
   - Menyimpan pesan ke list MESSAGES.
   - Menulis pesan ke file log.
5. Server mengirim respons: OK stored

6. Client menerima respons dan menampilkan ke layar.
7. Koneksi ditutup.
8. Server kembali ke mode listen.

B. Ambil Riwayat (HISTORY)

1. Client melakukan connect().
2. Client mengirim: HISTORY [n]
3. Server mengambil n pesan terakhir dari list.
4. Server mengirim daftar pesan ke client.
5. Client menampilkan riwayat.
6. Koneksi ditutup.

Jika belum ada pesan, server mengirim: EMPTY no messages ye

C. Bantuan (HELP)

Client mengirim: HELP

Server membalas daftar perintah yang tersedia.

III. Penjelasan Setiap Fungsi Utama

A. Server

1. main()

   Fungsi utama server yang:
   - Membaca argumen host, port, dan log file.
   - Membuat socket TCP.
   - Melakukan bind().
   - Melakukan listen().
   - Melakukan accept() dalam loop.
   - Membuat thread baru untuk setiap client.

   Fungsi penting:
   - bind() → Mengikat alamat IP dan port.
   - listen() → Mode standby.
   - accept() → Menerima koneksi masuk.
2. handle_client(conn, addr, log_path)

   Fungsi ini:
   - Menerima data dari client dengan recv().
   - Decode UTF-8.
   - Parsing perintah (MSG, HISTORY, HELP).
   - Memproses sesuai perintah.
   - Mengirim respons dengan sendall().
   - Menutup koneksi.

   Fungsi ini dijalankan dalam thread agar server bisa menangani banyak client.
3. log_to_file()

   Fungsi untuk:
   - Membuka file log.

- Menuliskan pesan ke file.
- Menyimpan aktivitas server.

Ini termasuk pengembangan tambahan (nilai tambah) berupa logging aktivitas.

4. format_messages()

   Menyusun format pesan menjadi: [YYYY-MM-DD HH:MM:SS] username (IP:PORT): message

B. Client

1. send_command()

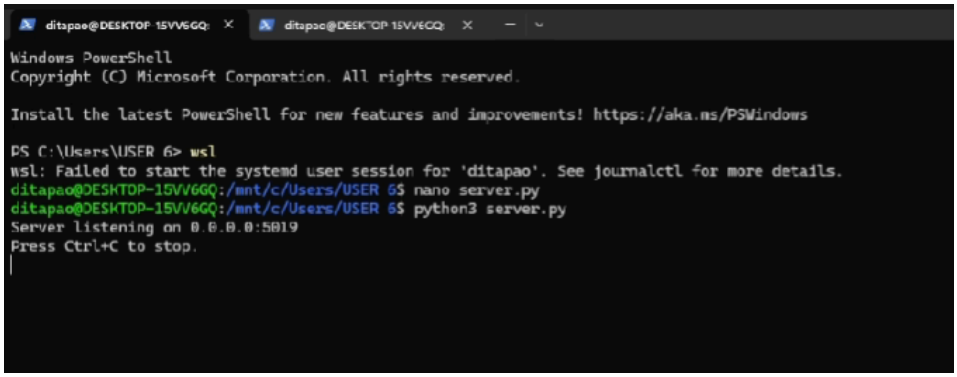
   Proses:
   - Membuat socket.
   - connect() ke server.
   - sendall() perintah.
   - recv() respons.
   - Menutup koneksi.
   - Mengembalikan hasil ke fungsi main.
2. main()

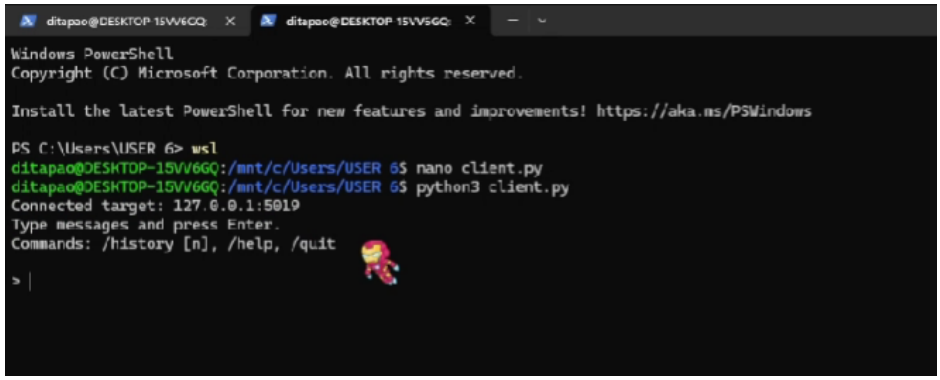
   Fungsi utama client:
   - Membaca argumen host, port, dan username.
   - Menampilkan prompt.
   - Mengubah input user menjadi format perintah server.
   - Menampilkan respons server.

   Perintah user:
   - /history
   - /help
   - /quit
   - Pesan biasa → otomatis menjadi MSG.

IV. Screenshot Eksekusi

- **Server sedang running dan listening pada port.**





- **Client berhasil connect ke server, mengirim pesan dan menerima respons OK, dan menampilkan history (riwayat pesan).**

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/P
SWindows

PS C:\Users\USER 6> wsl
ditapao@DESKTOP-15VV6GQ:/mnt/c/Users/USER 6$ python3 client.py --user dita
Connected target: 127.0.0.1:5000
Type messages and press Enter.
Commands: /history [n], /help, /quit

> hai
OK stored
> kamu tau jargon elins ga?
OK stored
> apaaa coba? elins???
OK stored
> /history
HISTORY
[2026-03-03 20:38:25] user (127.0.0.1:43198): halo
[2026-03-03 20:38:28] user (127.0.0.1:43204): /stored
[2026-03-03 20:41:41] andita (127.0.0.1:59352): hai
[2026-03-03 20:41:45] renata (127.0.0.1:38968): halo
[2026-03-03 21:12:28] dita (127.0.0.1:51688): hai
[2026-03-03 21:13:08] wan (127.0.0.1:43984): hai juga
[2026-03-03 21:13:16] dita (127.0.0.1:34196): kamu tau jargon elins ga?
[2026-03-03 21:13:20] wan (127.0.0.1:34210): tau dong
[2026-03-03 21:13:30] dita (127.0.0.1:54982): apaaa coba? elins???
[2026-03-03 21:13:35] wan (127.0.0.1:55362): one elins one family
> /help
COMMANDS
  MSG <username> <message>   store a message
  HISTORY [n]                get last n messages (default 10)
  HELP                       show this help
> /quit
Bye.
ditapao@DESKTOP-15VV6GQ:/mnt/c/Users/USER 6$
```

V. Analisis Kesalahan yang Mungkin Terjadi

Beberapa kesalahan yang mungkin terjadi pada sistem client–server ini adalah:

1. Server Belum Berjalan / Port Salah
   Client gagal terhubung dan muncul ConnectionRefusedError.
   - Penyebab: server belum dijalankan atau port/IP salah.
   - Solusi: pastikan server aktif dan port sesuai.
2. Format Perintah Salah
   Server mengirim pesan error seperti: ERROR usage: MSG <username> <message>
   - Penyebab: input tidak sesuai protokol.
   - Solusi: pastikan format perintah benar (MSG, HISTORY, HELP).
3. Koneksi Terputus
   Muncul error seperti ConnectionResetError atau WinError 10053.
   - Penyebab: client/server ditutup paksa atau jaringan terputus.
   - Solusi: jalankan ulang client dan pastikan koneksi stabil.
4. Data Hilang Setelah Server Restart
   Pesan disimpan di RAM sehingga hilang saat server mati.
   - Solusi: simpan data ke file atau database jika ingin permanen.
5. Port Sudah Digunakan
   Error Address already in use.
   - Penyebab: port belum dilepas oleh sistem operasi.
   - Solusi: gunakan SO_REUSEADDR atau tunggu beberapa saat sebelum restart.

VI. Refleksi Pemahaman

Dari tugas ini, kami memahami bahwa:

1. Proses komunikasi TCP melibatkan:
   - bind()
   - listen()
   - accept()
   - connect()
   - send()
   - recv()
   - close()
2. Client dan server harus memiliki protokol komunikasi yang jelas (MSG, HISTORY, HELP).
3. Encoding dan decoding UTF-8 penting agar data string dapat dikirim dengan benar.
4. Threading memungkinkan server melayani banyak client secara paralel.
5. Validasi input dan error handling sangat penting dalam sistem jaringan.
6. Logging membantu proses debugging dan monitoring aktivitas sistem.

Lampiran

Source Code:
https://drive.google.com/drive/folders/1G18xmwbIHs-Si-COn7rY_l2y8IT_cB8E?usp=drive_link